

Business Bench

Evaluating Agents on Business Deliverables

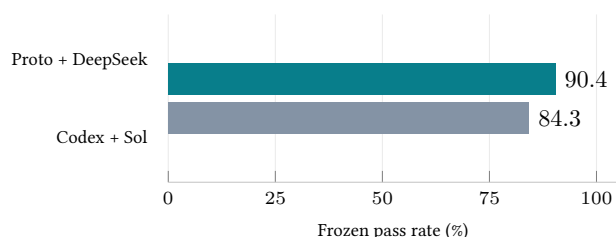
Somesh Misra Somnath Misra Shashank Dixit

ERP AI Research

Code, tasks, scorer, and evaluation evidence

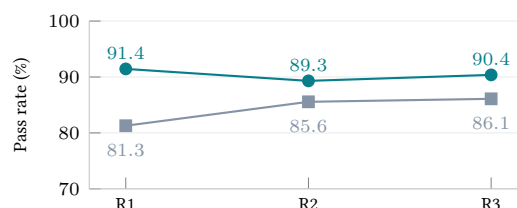
Abstract. Business delegation requires more than a high average score: a delivered artifact must satisfy its acceptance conditions, and success must survive repeated execution. We introduce Business Bench, an evaluation package built around explicit delivery contracts, and analyze a complete comparison on 187 desk tasks with three attempts per configured system. Proto with DeepSeek V4.1 Flash passes 507/561 attempts (90.4%); Codex with GPT-5.6-sol passes 473/561 (84.3%). Mean within-attempt check completion is higher, at 97.3% and 94.4%, while all-three acceptance falls to 78.6% and 75.4% of tasks. These quantities answer different operational questions. The aggregate difference is concentrated in reports and drafting; changing from the original evaluator to a shared retrospective scorer changes the between-system gap by 3.21 percentage points on the same outputs. We distinguish acceptance under a declared contract from the validity of that contract, execution repeatability from generalization, and reproducible accounting from independent artifact audit. The release supplies public tasks, executable scoring, an attempt ledger, and 20 unscored application-building tasks. Its results characterize two configured systems on a development-exposed workload, rather than identify separate model or harness effects.

A Overall task success



Captured-usage estimates: \$22.06 / \$219.13.

B Success by repetition



Teal: Proto + DeepSeek. Slate: Codex + Sol.

Figure 1. Complete desk comparison under the shared frozen scorer. Panel A summarizes all 561 attempts per system; panel B shows the three complete 187-task repetitions. All 1,122 attempts are retained. Costs are API-equivalent estimates, not invoices.

1. Introduction

Delegation transfers responsibility for an outcome. A reconciliation must support a payment decision; an import must preserve the records its destination needs; a changed application must retain its earlier permissions. A fluent explanation, a correct aggregate, or successful execution of most steps can coexist with an unusable handoff. The measurement problem is to specify the boundary at which the recipient can accept the work.

Business Bench makes this boundary explicit through a **delivery contract**: the requested artifact or state, its required properties, and the tests used to decide acceptance. The contract is conjunctive when all its requirements are necessary. This changes the interpretation of partial credit. If an import has the right schema but omits a required customer, averaging those two properties does not measure how often a recipient obtains an acceptable import.

An executable contract is nevertheless a measurement instrument, not business correctness itself. Tests may reject valid representations, accept invalid ones, or omit requirements. Repeating an execution can expose instability without testing new inputs. Hashing an evaluator can preserve its identity without establishing its validity. A useful evaluation must make these distinctions observable rather than compress them into a single score.

This report contributes a public workload of heterogeneous business artifacts; an accounting protocol separating acceptance, repetition, execution status, evaluator version, and resources; and a complete observational comparison

that exposes the consequences of those choices. The empirical questions are: how large is the gap between required-check completion and complete acceptance, where does the measured difference between systems arise, and how sensitive is that difference to evaluation design? Figure 1 gives the aggregate comparison. The rest of the paper explains what it measures and why the aggregate alone is insufficient.

2. Benchmark design

2.1 Acceptance as a declared contract

For task t , let x_t be the supplied fixture and instructions, and let a_{htr} be the artifact produced by configured system h on repetition r . A scorer G contains m_t required Boolean predicates g_{tj} . Its acceptance verdict is

$$Y_{htr}(G) = \prod_{j=1}^{m_t} g_{tj}(x_t, a_{htr}).$$

The product expresses logical conjunction; it makes no statistical independence assumption about the predicates. A predicate can encode a tolerance, a coverage threshold, or several related rules. Accordingly, $Y = 1$ means **accepted by the declared tests**, not that every property of the deliverable has been proved. Section 5 illustrates this distinction with an actual contract.

Conjunction also makes the acceptance boundary less sensitive to how checks are counted. Duplicating a predicate, or splitting one into logically equivalent conjuncts, preserves acceptance but can change the fraction of checks passed. Partial-check scores remain useful diagnostics; their units depend on evaluator decomposition. An acceptance verdict has a clearer handoff interpretation when every required property is genuinely necessary.

2.2 Desk workload and its population

The desk track contains 187 tasks. Inputs include CSV and XLSX exports, text and scanned documents, contextual messages, and occasional databases. Tasks require reconciliation of inconsistent records, application of supplied policies, preservation of identifiers and fields, and separation of supported conclusions from missing evidence.

Category	Tasks	Representative output
Spreadsheet	41	Reconciled or calculated workbook
Bookkeeping	36	Reconciliation, schedule, classification
Reports	30	Data summary and supported memo
Reformatting	26	Destination-compatible import file
Extraction	26	Structured records from documents
Drafting	19	Business text preserving supplied facts
Tooling	9	Small file-based tool or static page
Total	187	Three attempts per reported system

Table 1. Desk workload. Each task supplies an ask, workspace, generator, checks, and reference solution. Checks and references remain outside the evaluated agent’s container.

A task template, a fixture instance, and an execution repetition are different experimental units. Templates define a business problem and its rules; generators instantiate input data; repetitions execute a system again on the supplied fixture. This comparison repeats the same released task inputs. It measures execution repeatability on those inputs, not robustness across newly generated instances or unseen templates.

The fixtures are authored business-shaped data, not private customer engagements or a probability sample of business demand. The suite therefore defines an explicit empirical workload. Its category weights are design choices, and development exposure limits its use as an estimate of generalization.

2.3 Application and process extensions

The 20 build tasks extend the handoff to a running application: initial delivery followed by three changes to the same workspace. Their shared baseline covers roles, server-side scope, data integrity, validation, audit metadata,

and persistence. The runner starts a new one-shot turn for each change; the retained workspace provides continuity. No completed build leaderboard is reported.

The repository also develops a process track in which agents act within an ERP over multiple business dates. Its pilot is reported separately. The empirical claims in this paper concern the complete desk comparison dated 16 September 2026; neither released build specifications nor later process experiments are pooled into that score. These extensions broaden the objects a contract can address, but do not establish transfer of desk performance.

2.4 Position relative to prior work

WorkArena [1] evaluates enterprise software interaction in ServiceNow; SpreadsheetBench [2] evaluates spreadsheet manipulation and variation across test cases; TheAgentCompany [3] models work in a simulated company. EnterpriseClawBench [4] recovers tasks from workplace sessions, a provenance advantage that our authored fixtures do not claim. These settings differ in artifact type, environment, task origin, and evaluation coverage; they are not interchangeable measures of one ability.

The distinction between occasional success and repeated success also precedes this work: tau-bench [5] uses a pass-to-the-power-k reliability measure and evaluates resulting state. Executable evaluation of produced work is established practice, including software changes in SWE-bench [6]. Our contribution is the workload and its insupportable delivery contracts, together with analysis of how contract granularity, task composition, repetition, and retrospective scoring affect the conclusions.

3. Evaluation protocol

3.1 Configured systems and experimental boundary

The comparison covers all 187 desk tasks three times per system: **561 attempts per system and 1,122 attempts in total**. Every repetition is retained. Proto uses runtime revision c8f62dd60, DeepSeek V4.1 Flash, high reasoning, temperature 0, and DeepSeek-only routing without fallback. Codex uses CLI 0.154.0, GPT-5.6-sol, and high reasoning; its CLI-managed sampling is not asserted to match Proto’s temperature.

A configured system includes its model, harness, tools, skills, runtime, provider route, and resource limits. These factors vary together here. The Proto cohort combines one full repetition and two additional full repetitions of the same runtime; Codex’s three repetitions are reused from its completed cohort. The task sets and scorer are shared, but timing is not randomized or contemporaneous. The comparison identifies an observed system-level difference under these conditions, not the causal effect of a particular model, harness, or skill.

3.2 Estimands and evaluator chronology

For $T = 187$ tasks and $K = 3$ repetitions, attempt acceptance and all-three acceptance are

$$\widehat{P}_h = \frac{1}{TK} \sum_{t=1}^T \sum_{r=1}^K Y_{htr}, \quad \widehat{R}_h^{(K)} = \frac{1}{T} \sum_{t=1}^T \prod_{r=1}^K Y_{htr}.$$

The first measures accepted attempts in the complete matrix. The second gives a task credit only if every observed repetition passes. Equal repetitions make the first equivalent to the mean task-level acceptance rate. Neither quantity weights tasks by business value or estimates the distribution of work in a particular organization.

All primary verdicts use **conservative-v7**, a shared scorer applied to retained outputs. It strips workbook caches before native recalculation and includes equivalence checks for alternative representations. A receipt binds each verdict to its original result hash, output hashes, and scorer manifest. Original verdicts remain separate.

The chronology matters: the frozen scorer was constructed by the organization developing Proto after outputs existed. Freezing supplies a stable retrospective evaluation rule; it does not retroactively preregister that rule or make its selection independent of observed outputs. Section 6 quantifies the resulting sensitivity. There are four original grader-error attempts for Proto and three for Codex; the frozen scorer reports none.

Measure	Proto + DeepSeek V4.1 Flash	Codex + GPT-5.6-sol
Repetition 1	171/187	152/187
Repetition 2	167/187	160/187
Repetition 3	169/187	161/187
Frozen score	507/561 (90.4%)	473/561 (84.3%)
All three attempts pass	147/187	141/187
Original raw score	447/561 (79.7%)	431/561 (76.8%)

Table 2. All attempts under the frozen and original evaluators. Every frozen passing artifact also completed normally. Proto has three timeouts overall; Codex has none. Execution status and artifact acceptance are recorded separately.

3.3 Complete comparison and uncertainty

The frozen difference is **34 accepted attempts, or 6.06 percentage points**. The recorded paired bootstrap resamples task identifiers jointly across systems, retaining each task’s three repetitions: 20,000 resamples, seed 20260916, descriptive 95% interval +1.25 to +11.05 points. Resampling individual attempts instead would discard the task grouping that motivates this analysis.

The interval describes variation under this empirical task-resampling scheme. It does not incorporate scorer selection, related task families, development exposure, or a different deployment workload. Nor does a 90.4% aggregate imply 90% in every repetition: Proto’s second repetition is 167/187, or 89.3%.

4. Category results and repeatability

4.1 The aggregate difference is concentrated

Category	Tasks	Proto + DeepSeek	Codex + Sol	Gap (pp)
Bookkeeping	36	99/108 (91.7%)	98/108 (90.7%)	+0.9
Drafting	19	49/57 (86.0%)	37/57 (64.9%)	+21.1
Extraction	26	70/78 (89.7%)	69/78 (88.5%)	+1.3
Reformatting	26	74/78 (94.9%)	78/78 (100.0%)	-5.1
Reports	30	81/90 (90.0%)	56/90 (62.2%)	+27.8
Spreadsheet	41	115/123 (93.5%)	109/123 (88.6%)	+4.9
Tooling	9	19/27 (70.4%)	26/27 (96.3%)	-25.9

Table 3. Frozen results by category, including all three repetitions. Gaps are Proto minus Codex. The published aggregate weights categories by their task counts.

Reports contribute 25 additional accepted attempts and drafting contributes 12, together exceeding the net advantage of 34. Across the other 138 tasks, Proto accepts **377/414 (91.06%)** and Codex **380/414 (91.79%)**. This is a post hoc concentration diagnostic, not a replacement leaderboard: it shows that the aggregate difference is not a uniform advantage across the suite. Codex leads on reformatting and tooling; the largest positive gaps are in reports and drafting, which combine numerical and textual requirements. Their coverage needs independent validation.

For category acceptance rates $\widehat{P}_{hc}$ and nonnegative weights summing to one, the workload-specific difference is

$$\widehat{\Delta}(w) = \sum_c w_c (\widehat{P}_{\text{Proto},c} - \widehat{P}_{\text{Codex},c}).$$

The published weighting uses task shares; giving each of the seven categories equal weight changes the gap from 6.06 to 3.55 points. Neither weighting is an estimate of business demand. Because category differences have opposite signs, a ranking is conditional on the workload mixture. Deployment decisions additionally require the relevant severity, review cost, and service constraints, none of which this aggregate supplies.

Passing attempts per task	Proto + DeepSeek	Codex + Sol
3 of 3	147	141
2 of 3	29	17
1 of 3	8	16
0 of 3	3	13

Table 4. Tasks with zero, one, two, or three accepted attempts. Each column sums to 187; weighting the rows by accepted attempts recovers 507 and 473.

4.2 Repeated success is a separate property

Proto passes all three attempts on **147/187 tasks (78.6%)**, versus **141/187 (75.4%)** for Codex. At least one attempt passes on 184 and 174 tasks, respectively; 37 Proto tasks and 33 Codex tasks have mixed outcomes. At-least-one success describes observed availability with hindsight. It is not the performance of a user who must identify the successful artifact without access to the grader.

Under conditionally independent repetitions with a common task-specific success probability p_{ht} , expected all-three acceptance is $E_t[p_{ht}^3]$. By convexity, this is at least $(E_t[p_{ht}])^3$, with equality for constant task probabilities. Cubing a pooled rate therefore generally misses task heterogeneity. The reported statistic counts observed all-three outcomes directly and assumes no independence of the actual executions. Three observations per fixed fixture do not establish a deployment failure probability.

The task-and-repetition cross-tabulation contains 431 pairs where both pass, 76 where only Proto passes, 42 where only Codex passes, and 12 where neither passes. Repetition identifiers permit bookkeeping, not matched randomness: they are neither common seeds nor simultaneous trials. Consequently these pairs should not be treated as 561 independent experimental blocks.

4.3 Partial credit and the acceptance boundary

Let $C_{ht,r,j}$ denote required predicate j 's pass indicator. An attempt-weighted partial-contract score is

$$\widehat{Q}_h = \frac{1}{TK} \sum_{t,r} \frac{1}{m_t} \sum_{j=1}^{m_t} C_{ht,r,j}, \quad \widehat{P}_h \leq \widehat{Q}_h.$$

The inequality follows per attempt: the product of Boolean predicates cannot exceed their mean. It requires no assumption about why failures occur. It also identifies a precise comparison: pooling all predicates would additionally weight tasks by their number of checks, which ranges from three to twelve in this suite.

Measure	Proto + DeepSeek	Codex + Sol
Complete attempts accepted	507/561 (90.37%)	473/561 (84.31%)
Mean within-attempt check fraction	97.34%	94.42%
Pooled required checks passed	3,698/3,801 (97.29%)	3,558/3,801 (93.61%)
Failed attempts with one failed required check	34/54 (63.0%)	39/88 (44.3%)

Table 5. Acceptance and partial-contract diagnostics computed from the frozen ledger. The mean check fraction weights attempts equally; the pooled fraction weights check instances equally. The last row conditions on failed attempts and therefore uses a different denominator.

Mean within-attempt check completion exceeds full acceptance by 6.96 points for Proto and 10.10 for Codex. Many rejected artifacts satisfy most declared checks. That can be useful for diagnosis, yet it does not make a required omission acceptable to the recipient.

Exactly one required predicate fails in 34 of Proto's 54 failures and 39 of Codex's 88. This is a statement about predicate outcomes, not causal attribution or repair effort. A custom predicate can bundle several business rules; one failed predicate need not mean one underlying mistake. Its importance is that high component scores can conceal a recurring failure to complete the whole contract.

5. Worked example: payment reconciliation

5.1 Inputs and required handoff

The payments-match-v2 task supplies `bank_export.csv`, `open_invoices.csv`, and `note.txt`. The owner requests three CSV files; the note states the policy. Reference outputs and grader code are withheld from the agent container.

Deliverable	Required fields	Business purpose
unpaid.csv	invoice_id, customer, amount_outstanding	Preserve remaining open balances
matches.csv	invoice_id, line_id, amount_applied	Identify the bank lines settling invoices
unapplied.csv	line_id, amount, reason	Retain credits that cannot be applied

Table 6. A three-part contract. An accurate total cannot substitute for the required relationships between invoices, bank lines, and remaining balances.

5.2 Context determines correctness

Stripe remittances arrive net of 2.9% plus 30 cents, and the note directs the agent to mark their invoices paid in full. International wires short by up to about 2% receive the same treatment for intermediary charges. A genuine 60% partial payment instead leaves a 40% balance. Identical arithmetic differences can therefore require different accounting decisions.

One credit names another customer’s invoice; the note gives payer and amount precedence over that reference. A payment identifying an already-closed invoice, however, must remain unapplied. A named multi-invoice ACH and a customer-only wire must be distributed across the invoices they cover, while an exported duplicate must not be counted twice. Supplier refunds and interest remain unapplied; debits are excluded.

These cases test policy-conditioned relationships, not merely extraction or arithmetic. They also illustrate why a general preference such as “trust the reference” cannot replace reading the task’s declared rule.

5.3 What acceptance does and does not establish

Structural checks enforce columns and exact sets of open and unapplied identifiers; outstanding balances have a one-cent tolerance. The matching check examines applied amounts and contributing bank-line identities. Named duplicate, multi-invoice, fee, wrong-reference, refund, and closed-invoice cases have additional requirements.

The general paid-invoice matching predicate nevertheless accepts at least **90% agreement**. All required predicates passing therefore does not imply every row is correct. A threshold inside a predicate remains part of the contract even when aggregation across predicates is strict. Both systems pass this task in all three repetitions. The example explains the evaluation boundary; it does not explain their aggregate difference.

6. Scoring integrity and evaluator sensitivity

6.1 Same outputs, different verdicts

Original verdict → Frozen verdict	Proto + DeepSeek	Codex + Sol
Pass to pass	447	430
Fail to pass	56	40
Ungraded to pass	4	3
Pass to fail	0	1
Fail to fail	54	87

Table 7. Original-to-frozen transitions over all 561 attempts per system. Ungraded means an original grader error. Rescoring evaluates retained outputs; it is not an agent rerun or artifact repair.

Proto gains 60 passes: 56 original failures and four ungraded attempts. Codex gains a net 42: 40 failures and three ungraded attempts become passes, while one original pass becomes a failure. Its original 431 passes thus include one that the frozen evaluator rejects.

The original gap is $100(447 - 431)/561 \approx 2.85$ points; the frozen gap is $100(507 - 473)/561 \approx 6.06$ points. The change in the between-system difference is

$$100 \frac{(507 - 447) - (473 - 431)}{561} \approx 3.21 \text{ percentage points.}$$

The additional 18-pass difference comes entirely from evaluator changes applied to the same outputs. This is material measurement sensitivity. It is neither evidence of agent improvement nor, by itself, proof that the revisions are biased: valid alternative representations can deserve acceptance. Determining which verdict better reflects the business requirement needs independent adjudication.

6.2 Reproducibility and validity are separate

A common scorer removes evaluator-version differences between the primary system scores. Frozen source and receipts make that comparison traceable and make later changes detectable against the recorded scorer fingerprint. They do not establish that the rule was selected without knowledge of the systems' outputs. This distinction is especially consequential because the benchmark publisher also develops Proto and the retrospective revisions increase its measured lead.

There are three levels of evidence. **Arithmetic verification** recomputes the released matrix and aggregates from the public ledger. **Evaluator inspection** examines the released source and its declared predicates. **Independent artifact audit** requires access to actual retained outputs and an external acceptance judgment. Public hashes support identity checks if those artifacts become available, but cannot reconstruct them. The current public release supports the first two levels; private raw outputs limit the third.

6.3 Validating the measurement instrument

Reference solutions passing and untouched workspaces failing are useful positive and negative controls. They do not estimate false acceptance or false rejection on plausible agent outputs. Untouched inputs are often an easy negative case; a realistic invalid artifact may satisfy every checked property while violating an omitted rule.

A stronger validation design would use blinded practitioner judgments over a stratified sample of accepted and rejected artifacts, adjudicate disagreements, and report error rates with sampling uncertainty. Targeted invalid mutations would probe specific boundaries such as identifier substitution, duplicated settlements, or absent authorization. These are proposed tests, not completed evidence. Future confirmatory campaigns should freeze and review the evaluator before running held-out fixtures, while retaining original verdicts whenever scoring is revised.

7. Resource use and the cost of accepted work

Measure	Proto + DeepSeek	Codex + Sol
Input tokens	823,587,929	165,959,457
Cached input	775,672,448	150,329,088
Uncached input	47,915,481	15,630,369
Output tokens	20,910,907	4,823,643
Estimated model cost (USD)	22.06	219.13
Median task duration (s)	124.9	135.2
p90 task duration (s)	455.6	403.3
Summed task duration (s)	113,460.1	111,218.8

Table 8. Resources for all 561 attempts per system. Costs are captured-usage API-equivalent estimates at the recorded price table, not subscription invoices or reconciled provider bills. Summed durations are occupied task time, not elapsed campaign time under concurrency.

Dividing total captured estimated model cost by accepted artifacts gives **\$0.0435 for Proto** and **\$0.4633 for Codex**. This ratio allocates the observed model spending on failures to the accepted outputs. It excludes verification labor, repairs, infrastructure, and missing usage. It is an ex-post campaign accounting ratio, not an expected cost for retrying until success: such a policy would require a way to recognize acceptance and a model of dependence across retries.

Proto has lower estimated model cost and median duration but uses more tokens, slightly more summed task time, and a worse p90. Its price advantage is therefore not uniform dominance in resource use. Cache fractions are token-weighted, and uncached totals are reported separately.

Proto records 16,954 completed model responses; Codex records 561 task-level usage aggregates, not comparable request counts. Nine Proto request starts lack completed responses, so their additional usage is unknown. Separate Codex reasoning-token counts, first-token latency, and turn-start versus intra-turn cache splits are unavailable. These omissions bound the accounting claims.

8. Validity, generalization, and decisive next experiments

Construct validity. The target is useful delegated work; the observation is acceptance by authored tests. Model-assisted authoring, planted truth, and deterministic checks do not close that gap. Practitioner review should assess specification quality, human baselines should calibrate difficulty, and deployment-level claims require a defined workload sampling frame.

Identification. Model, harness, provider, tools, skills, sampling, and cohort dates vary together. A same-model harness comparison would need those factors controlled; a model comparison would need a fixed harness and matched conditions. This release identifies neither effect separately.

Generalization. The suite was used during development, includes related task families, and repeats fixed fixtures. New instances of known templates, held-out templates, and deployment workloads are successively different targets. The reported bootstrap does not supply evidence across these boundaries. A decisive next evaluation would use sealed fixtures and templates with a reviewed scorer fixed before runs, and report results separately for each form of novelty.

Operational validity. Binary acceptance does not price failure severity, reversibility, detection, or downstream damage. A small residual balance error and an authorization failure need not have comparable loss. Build-track acceptance requires direct tests of permissions, persistence, and regressions after changes; a reachable URL or a filled tester sheet is insufficient. No such completed build comparison is claimed.

Reproduction boundary. The public package releases source, task inputs, hashes, and verdict records. Historical agent environments, credentials, traces, and raw generated artifacts are not fully released. A new configured run is possible with the required access; exact reconstruction or independent re-adjudication of every historical output is not. The appendices state the evidence retained and the prospective record needed for a new campaign.

9. Conclusion

Business Bench makes acceptance at the handoff an explicit measurement object. The complete desk comparison reports 90.4% versus 84.3% accepted attempts, yet the more consequential result is the structure behind those rates: required-check completion exceeds full acceptance, all-three acceptance is lower still, the aggregate advantage is concentrated in two categories, and retrospective evaluator changes materially alter its magnitude.

These findings support separate reporting of acceptance, repeatability, workload composition, evaluator sensitivity, and cost. They do not identify a generally superior model or harness, or establish autonomous readiness for business operations. The next evidential step is an independently reviewed contract tested prospectively on sealed work, with sufficient artifact access to audit disagreement between measured acceptance and business judgment.

References

- [1] Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H. Laradji, Manuel Del Verme, Tom Marty, Léo Boisvert, Megh Thakkar, Quentin Cappart, David Vazquez, Nicolas Chapados, and Alexandre Lacoste. WorkArena: How capable are web agents at solving common knowledge work tasks? *arXiv preprint arXiv:2403.07718*, 2024. doi: 10.48550/arXiv.2403.07718. URL <https://arxiv.org/abs/2403.07718>.
- [2] Zeyao Ma, Bohan Zhang, Jing Zhang, Jifan Yu, Xiaokang Zhang, Xiaohan Zhang, Sijia Luo, Xi Wang, and Jie Tang. SpreadsheetBench: Towards challenging real world spreadsheet manipulation. *arXiv preprint arXiv:2406.14991*, 2024. doi: 10.48550/arXiv.2406.14991. URL <https://arxiv.org/abs/2406.14991>.
- [3] Frank F. Xu, Yufan Song, Boxuan Li, Yuxuan Tang, Kritanjali Jain, Mengxue Bao, Zora Z. Wang, Xuhui Zhou, Zhitong Guo, Murong Cao, Mingyang Yang, Hao Yang Lu, Amaad Martin, Zhe Su, Leander Maben, Raj Mehta, Wayne Chi, Lawrence Jang, Yiqing Xie, Shuyan Zhou, and Graham Neubig. TheAgentCompany: Benchmarking LLM agents on consequential real world tasks. *arXiv preprint arXiv:2412.14161*, 2024. doi: 10.48550/arXiv.2412.14161. URL <https://arxiv.org/abs/2412.14161>.
- [4] Jincheng Zhong, Weizhi Wang, Che Jiang, Kai Tian, Zhenzhao Yuan, Junlin Yang, Dianqiao Lei, and Kaiyan Zhang. EnterpriseClawBench: Benchmarking agents from real workplace sessions. *arXiv preprint arXiv:2606.23654*, 2026. doi: 10.48550/arXiv.2606.23654. URL <https://arxiv.org/abs/2606.23654>.
- [5] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. tau-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024. doi: 10.48550/arXiv.2406.12045. URL <https://arxiv.org/abs/2406.12045>.
- [6] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? *arXiv preprint arXiv:2310.06770*, 2023. doi: 10.48550/arXiv.2310.06770. URL <https://arxiv.org/abs/2310.06770>.

A. Running and auditing the benchmark

A.1 Installation and execution

Use Python 3.11 or 3.12 and Docker on a dedicated Linux evaluation host. Install requirements.txt and build the agent and recalculation images with docker/build.sh. Provision a benchmark-owned model login; private agent runtimes are supplied separately. The README contains full commands for each supported adapter and both tracks.

The desk runner mounts only the workspace, isolated home, and read-only adapter into the agent container. References remain on the host. Local-process mode is for development: it does not isolate host-side answers. Network access remains enabled for model requests and permitted tooling; containers are not a guarantee against online discovery of an exposed task set.

Use a fresh result label for each run. The runner refuses to overwrite existing desk labels or attempt directories. Build runs require an explicitly allocated port and a dedicated test environment. Do not expose unreviewed generated applications or mount production credentials. Build tester sheets include generated logins and must remain private.

A.2 Scoring and verification

```
python -m unittest discover -s tests -v
python bench/export_campaign.py --verify
python bench/release_manifest.py
python scoring/frozen-v7/scorer.py TASK_ID WORKSPACE
```

Set BENCH_RECALC_DOCKER_IMAGE to the native recalculation image when scoring workbooks. The last command uses the frozen scorer directly; bench/run.py records the original grader's result. Preserve both fields when publishing a new frozen-scored campaign. Never silently substitute a subset of rescored attempts into a raw aggregate.

The canonical manifest digest is:

```
b4720db00f2a55461ca70767a4baf8dd2d25aa9d301e8fe22ea13d21ccca358f
```

The package preserves its original assembly-status text to keep its fingerprint immutable; the subsequent recorded replay covered 1,341 attempts with zero verdict differences. Current export verification checks the receipt and source hashes rather than editing that historical manifest.

A.3 Release validation

All 187 desk reference-solution/untouched-workspace checks passed with native recalculation; all 20 build-task packs passed structural and seed validation. The strict desk validator reported 28 tasks whose supplied fixture bytes differed from regeneration despite deterministic repeated regeneration. The supplied inputs remain authoritative and unchanged. Affected tasks are listed in docs/validation.md; byte differences are not assumed semantically harmless.

B. Reproduction and metric definitions

B.1 Launching a new complete desk run

The following sequence installs the benchmark and launches the Codex desk matrix after a benchmark-owned login has been provisioned as described in README.md. It makes paid model calls. Use a dedicated Linux host, a unique label, and the exact intended adapter configuration; the example is a new experiment, not a reconstruction of a private historical agent environment.

```
python3.12 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
bash docker/build.sh
export BENCH_RECALC_DOCKER_IMAGE=bench-recalc:release
```

```
python bench/run.py --docker business-bench:release \
  --harness codex-sol --tasks all --runs 3 \
  --parallel 2 --label reproduction-codex
```

Start with a single-task smoke run under a different label before the full matrix. A different model, provider route, skill inventory, or runtime revision defines a different cell and must be named accordingly. Proto adapters require a separately supplied runtime and isolated model configuration; platform-backed build runs additionally require dedicated test-organization access.

B.2 Minimum reproducibility record

Record	Required content
Workload	Task identifiers, task/input hashes, repetition mapping
Agent environment	Runtime revision, image digest, model route, tools and skills
Sampling and budget	Reasoning setting, temperature where controllable, timeout, CPU and memory
Scoring	Frozen package digest, original and primary verdicts, grader errors
Evidence	Original result hash, output hashes, private artifact retention location
Accounting	Captured usage, price assumptions, durations, missing observations

Table 9. Minimum record for a new campaign. Missing fields must be disclosed rather than inferred from a successful result. These are prospective requirements, not a claim that every historical environment detail is reconstructible.

B.3 Formal definitions

Let Y_{htr} denote frozen acceptance, with $T = 187$ and $K = 3$. Sections 3 and 4 define attempt acceptance $\widehat{P}_h$, all-three acceptance $\widehat{R}_h^{(K)}$, and mean within-attempt check completion $\widehat{Q}_h$. Observed at-least-one acceptance and the pooled check fraction are

$$\widehat{A}_h^{(K)} = \frac{1}{T} \sum_t \mathbf{1} \left\{ \sum_r Y_{htr} > 0 \right\}, \quad \widehat{Q}_h^{\text{pool}} = \frac{\sum_{t,r,j} C_{htrj}}{K \sum_t m_t}.$$

The first uses hindsight over repeated attempts; the second weights tasks by their predicate counts. Neither is substituted for attempt acceptance. The repeated-success measure follows the pass-to-the-power-k interpretation used in tau-bench [5].

For captured input I , cached input C , output O , and prices per million tokens π_u, π_c, π_o , estimated model cost is

$$\widehat{K}_{\text{model}} = \frac{(I - C)\pi_u + C\pi_c + O\pi_o}{10^6}.$$

This expression applies to usage within a price cell; aggregate cost sums the corresponding estimates. Uncached input is total input minus cached input. Aggregate cache fraction is total cached input divided by total input, not the mean of request percentages. Missing usage is unknown, not zero.

Durations are attempt-level observations. The p90 uses nearest rank; summed durations can exceed elapsed campaign time under concurrency. Gaps between acceptance rates are percentage points, not relative error reductions.